Prompt Engineering: Unlock Your LLM's Full Potential

1. Hook & Intro

Side-by-side: Bad prompt vs. great prompt

Bad prompt:

```
Write something about our product.
```

Result: Generic marketing fluff, wrong tone, wrong length, no clear CTA.

Great prompt:

```
You are a senior B2B copywriter. Write a 2-sentence LinkedIn ad for our project-management SaaS (Asana alternative). Audience: ops managers at mid-size companies. Tone: confident but not salesy. End with a clear CTA to start a free trial. No emojis.
```

Result: On-brand, scoped, actionable, and easy to drop into an ad.

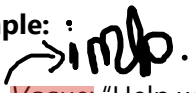
The same model produces both; the difference is **how you ask**.

> **"Most people are leaving 80% of an LLM's capability on the table."**

2. What Is Prompt Engineering & Why It Matters

Programming in natural language

- You're **giving instructions in plain language** instead of code.
- The model has no built-in "task list"—you define the task, role, format, and constraints in the prompt.
- The same model can seem **brilliant or useless** depending on clarity, context, and structure.

Example: 

- *Vague:* "Help with my essay." → Model doesn't know subject, length, or style.
- *Specific:* "You are a tutor. Help me improve the thesis and first paragraph of this 500-word history essay on the causes of WWI. Keep my voice; suggest edits inline." → Model can actually help.

Who benefits

- **Developers** — code generation, debugging, docs, refactors
- **Marketers** — copy, ads, emails, social, A/B ideas
- **Researchers** — summarization, lit review, brainstorming

- **Everyday users** — writing, planning, learning, decision-making

Takeaway: Better prompts = better results, for everyone. It's a skill that compounds.

---

3. Foundation: How LLMs "Think"

→ Transformers

Next-token prediction (intuition only)

- The model predicts the **next token** (word or sub-word) given everything so far in the prompt and conversation.
- It has **no memory** of past chats unless you include that information in the current context.
- So: **context, specificity, and structure** in your prompt directly shape what it says next.
- More relevant context and clearer instructions → more relevant and consistent outputs.

Steering vs. commanding

- **Commanding:** "Summarize this." → Model chooses length, style, and focus.
- **Steering:** "You are an executive assistant. Summarize this meeting transcript in 4 bullet points. Focus on decisions and action items. No filler." → You steer length, focus, and format.

Example:

Commanding	Steering
"Write an email."	"You are replying to a client who asked for a project delay. Write a short, apologetic email. Propose new deadline: March 15. Tone: professional and reassuring."

Steering (role, audience, format, constraints) gives more consistent, usable results.

---

4. Core Techniques → Better output

4a. Be specific & set the scene

Define **role**, **audience**, **tone**, and **format** so the model doesn't have to guess.

- **Role:** "You are a senior technical writer."
- **Audience:** "Writing for developers who know Python but not cloud APIs."
- **Tone:** "Professional but friendly; avoid jargon or explain it once."
- **Format:** "Start with a 2-sentence overview, then numbered steps, then a short 'Common pitfalls' section."

Example — support reply:

Weak:

```
Reply to this customer complaint.
```

Strong:

```
You are a customer support lead. Reply to this complaint from a paying user whose export failed twice. Acknowledge the frustration, apologize briefly, confirm we're investigating, and offer a concrete next step (e.g. we'll email within 24h). Keep it under 150 words. Sign off as "Support Team."
```

4b. Few-shot prompting

Give **2–4 examples** of input → output pairs. The model infers the pattern and replicates it.
Especially helpful for: **format**, **style**, **edge cases**, and **classification**.

Example — turning feedback into ticket titles:

Without examples (zero-shot):

```
Turn this user feedback into a short ticket title.
Feedback: "The app crashes when I upload a PDF over 10MB on iPhone."
```

Might get: "User reports crash" or something too long or inconsistent.

With examples (few-shot):

```
Turn user feedback into a short ticket title (under 60 chars). Use format: [Area] Brief description.

Feedback: "Login is broken with Google on Safari."
Title: [Auth] Google login fails on Safari

Feedback: "Export to CSV only exports first 100 rows."
Title: [Export] CSV export limited to 100 rows

Feedback: "The app crashes when I upload a PDF over 10MB on iPhone."
Title:
```

Model is much more likely to output: [Upload] Crash on large PDF (iPhone) or similar.

4c. Chain-of-thought (CoT)

Ask the model to **reason step by step** before giving the final answer.
Reduces errors on **logic**, **math**, **multi-step planning**, and **comparisons**.

Example — simple reasoning:

Without CoT:

```
A store sells pens for $2 and notebooks for $5. Sarah buys 3 pens and 2 notebooks.
She has a 10% off coupon. How much does she pay?
```

Model may jump to a number and sometimes make arithmetic mistakes.

With CoT:

```
A store sells pens for $2 and notebooks for $5. Sarah buys 3 pens and 2 notebooks.
She has a 10% off coupon. How much does she pay? Think step by step: find
subtotal, then apply discount, then state final amount.
```

Model is guided to: 3×2 + 2×5 = 16 → 10% off = 1.60 → 16 - 1.60 = 14.40.

Use phrases like: **"Think step by step."** / **"Show your reasoning."** / **"Explain each step before concluding."**

4d. Structured output

Ask explicitly for **JSON**, **tables**, **XML**, or **markdown** so you can parse and use the output in code or docs.

Example — product comparison:

Free-form:

```
Compare our product to Competitor X and Competitor Y on price, features, and
support.
```

You get prose; hard to turn into a table or app.

Structured:

```
Compare our product (OurApp) to Competitor X and Competitor Y. Respond with valid
JSON only, no other text, in this shape:

{
  "products": [
    { "name": "...", "price": "...", "keyFeatures": ["...", "..."], "support":
"..." }
  ]
}
```

You get machine-readable output you can validate and display.

4e. Constraints & negative instructions

Say what you **don't** want: length, tone, format, or topic.
Reduces: too long, too casual, wrong structure, or off-topic tangents.

Examples:

- **Length:** "Summarize in exactly 3 bullet points." / "Keep the reply under 100 words."
- **Tone:** "No slang or humor." / "Do not apologize."
- **Format:** "Do not use bullet points; use one short paragraph."
- **Scope:** "Do not suggest paid tools." / "Do not include code; describe the approach only."

Example — avoiding a bad habit:

Without constraint:
"Write a short intro for our onboarding doc."
Might start with "Welcome!" or marketing fluff.

With constraint:
"Write a short intro for our onboarding doc. Start directly with what the user will do in this section. Do not start with 'Welcome' or generic greetings."
More likely to be actionable from the first sentence.

4f. Iterative refinement

Treat prompting as a **conversation**, not a one-shot.
First reply not quite right? Refine: "Shorter." "More formal." "Add one example." "Focus only on X."

Example flow:

1. **You:** "Draft a 2-sentence blurb for our new feature: smart scheduling."
2. **Model:** [Returns 4 sentences, a bit salesy.]
3. **You:** "Cut it to 2 sentences and make it more factual, less salesy."
4. **Model:** [Tighter, more factual.]
5. **You:** "Add that it works with Google Calendar."
6. **Model:** [Final version.]

Iteration is normal and often faster than writing the perfect prompt on the first try.

4g. Interview-style prompting: let the model ask → Underrated / Powerfull

Instead of guessing what context to provide, **give the model the task and ask it to interview you** for whatever it needs. You state the goal; the model asks clarifying questions; you answer; then it performs with full context.

Why it works: We often leave out details (audience, constraints, format, examples) because we don't know the model needs them. Letting the model ask surfaces those gaps before it generates.

How to do it:

1. **Give the task.** Describe what you want in a sentence or two.

2. **Ask the model to interview you.** e.g. “Before you do this, interview me: ask me any questions you need answered to do this well. Ask one at a time. I’ll answer; when you have enough, say you’re ready and then do the task.”
3. **Answer each question.** Be concise but complete.
4. **Let the model proceed.** Once it has what it needs, it delivers the output.

Example — blog post:

You (turn 1):

```
I need a short blog post (about 400 words) for our company blog. Topic: why small teams should try async standups. Before you write it, interview me: ask me any questions you need to do this well—audience, tone, product mentions, format, etc. Ask one question at a time. When you have enough information, say "I have enough—here's the post" and then write it.
```

Model (turn 2): “Who is the primary audience for this post? (e.g. startup founders, engineering leads, remote teams?)”

You: “Engineering leads at small product teams (5–15 people), mostly remote or hybrid.”

Model (turn 4): “Should we mention a specific product or tool, or keep it product-agnostic?”

You: “We can mention our tool, StandupBot, once in the intro and once near the CTA. No hard sell.”

Model (turn 6): “Any tone preferences? (e.g. casual, data-driven, opinionated)”

You: “Friendly and practical, with one or two concrete tips they can try today.”

Model (turn 8): “I have enough—here’s the post.” [Then writes the post with the right audience, product mentions, and tone.]

When to use it: Complex or high-stakes tasks (e.g. strategy doc, customer email, pitch), when you’re unsure what context matters, or when you want to avoid multiple “wrong draft → refine” rounds. For trivial tasks, a single well-steered prompt is faster.

---

5. Advanced Strategies

System vs. user prompts (API context)

- **System prompt:** Sets identity, rules, and style (often not shown to end users). Use for “always-on” behavior.
- **User prompt:** The actual request. Keep it focused on this turn’s task.

Example:

- **System:** “You are a helpful coding assistant. You answer in concise, correct snippets. You never make up API names; if unsure, say so.”
- **User:** “How do I read the first line of a file in Python?”

The system prompt keeps tone and rules consistent across many user questions.

Prompt chaining

Break **complex tasks** into steps. Use the output of step N as input to step N+1.
Improves reliability and makes it easier to fix or improve one step at a time.

Example — blog post pipeline:

1. **Step 1:** "Given topic [X], output a 5-heading outline. JSON: { "headings": ["...", "..."] }."
2. **Step 2:** "Expand this outline into a 400-word section. Outline: [paste Step 1 output]. Tone: [Y]."
3. **Step 3:** "Turn this draft into meta title and description for SEO. Draft: [paste]. Max 60 chars title, 155 chars description."

Each step has one clear job and one clear output format.

Self-evaluation

Ask the model to **critique or score** its own output. Use for drafts, code, or summaries.

Example:
"Here's a short summary I generated: [text]. Rate it 1–5 for clarity and completeness. In one sentence, suggest the single most important improvement."

Use the critique to refine in the next turn.

Temperature & other parameters

- **Temperature:** Lower (e.g. 0.2) = more deterministic, repeatable; higher (e.g. 0.8) = more creative, varied.
- Use **low** for: facts, code, structured output, consistency.
- Use **higher** for: brainstorming, varied phrasing, multiple ideas.
- If outputs are too random or off-task, lower temperature. If too repetitive, raise it slightly.

---

6. Common Mistakes & How to Fix Them

Mistake	What goes wrong	Fix
Too vague	Generic or irrelevant output.	Add role, audience, tone, format, and length.
Too many tasks in one prompt	Model drops or mixes tasks.	Split into steps or separate prompts (chaining).
Not enough context or examples	Wrong format or style.	Add 1–3 few-shot examples; include relevant background.
Ignoring format	Hard to parse or reuse.	Explicitly request structure (e.g. "JSON with keys X, Y, Z").

Mistake	What goes wrong	Fix
Assuming memory	Model “forgets” earlier in the thread.	Repeat or summarize key facts in this turn if the thread is long.

Example of “too many tasks”:

Overloaded:
“Summarize this article, list the main criticisms, suggest 3 discussion questions, and write a tweet about it.”
Better: do “summarize” first, then “list criticisms,” then “discussion questions,” then “tweet,” each with clear instructions.

---

7. Real-World Use Case Walkthroughs

Writing & content

- **Blog post:** Role (e.g. content lead) + audience + 3–5 heading outline + tone + word count.
- **Email:** Purpose (e.g. follow-up) + recipient + key points + one CTA + length.
- **Ad copy:** Product, audience, platform, character limit, and “no X” (e.g. no emojis).

Example — email:

```
You are writing a follow-up email to a prospect who didn’t reply to your first
message. Product: project management tool for small teams. Goal: one short
paragraph that adds value (e.g. one tip or resource), then one soft CTA to reply.
Tone: helpful, not pushy. No guilt-tripping. Under 80 words.
```

Code & debugging

- Provide **code + language + framework**. Ask **one** focused thing per turn: explain, find bug, add error handling, refactor, or add tests.
- Example: “This Python function sometimes raises KeyError. [paste code]. Find the cause and suggest a minimal fix with a one-line comment explaining why.”

Data analysis & summarization

- Specify **output format**: “Summarize in 3 bullets.” / “Main trends in a table.” / “Compare X and Y in one short paragraph.”
- Example: “Here are last month’s support tickets by category. [data]. In a markdown table, show: category, count, % of total. Add one sentence on the biggest driver.”

Research & brainstorming

- Ask for **multiple options**: “List 10 ideas for …” / “Pros and cons of …” / “What am I missing?”
- Use slightly **higher temperature**; iterate with “expand on #3” or “more like #2, less like #5.”
- Example: “We’re naming a new feature: it suggests the best time to send emails. Give 5 name options. Short, memorable, no jargon. Then pick your top one and say why in one sentence.”

---

8. Prompt Engineering Toolkit & Resources

Prompt libraries and templates

- Browse **PromptBase**, **FlowGPT**, **OpenAI examples**, or **Anthropic's prompt library**.
- Save and adapt templates for your own roles and tasks (e.g. "support reply," "blog outline," "code review").

Official guides

- **Anthropic** — prompting docs, prompt design, and safety.
- **OpenAI** — prompt engineering guide and API best practices.
- **Other providers** — check their docs for model-specific tips and limits.

Habit: keep a "prompt journal"

- **Save** prompts that work well (and which model/settings you used).
- **Note** what you changed when something didn't work.
- **Build** a small personal library (e.g. "support," "outline," "summarize") and refine over time.

Wrap-up

- **Prompt engineering** = programming in natural language; small changes unlock most of an LLM's value.
- **Core:** Be specific (role, audience, tone, format), use few-shot examples, ask for reasoning (CoT) and structure (JSON/markdown), set constraints, iterate. For complex tasks, try **interview-style**: give the task and ask the model to interview you for what it needs before it performs.
- **Advanced:** Use system vs. user prompts, chain steps, add self-evaluation, tune temperature.
- **Avoid:** Vague prompts, overloading one prompt, no context/examples, assuming memory.
- **Practice:** Use real tasks (writing, code, data, research) and keep a prompt journal.

Next step: Pick one technique (e.g. "be specific" or "few-shot") and try it on a task you do every week. Save the prompt that works and one thing you improved.